

Decision Modeling for Predict–Detect–Simulate Logistics: A Pre-Factory-Tour Framework for DENSO Factory Hacks D2

Constraints, Objectives, Uncertainty, and Simulation-Based Recommendation

Phạm Quang Minh

Decision/Optimization/Mathematical Modeling role, DENSO Factory Hacks 2026 D2

September 2026

Abstract

This report is the normative specification of the decision layer of the D2 system (“Simulate & Forecast Logistics, Recommend Actions”) as it can be defined before any factory-specific evidence exists. The recommendation problem is stated over a candidate action set filtered by a three-valued (satisfied/violated/unknown) hard-constraint logic, evaluated through a discrete-event simulator under a shared scenario–seed bank, and aggregated by a preference rule that remains a human decision. Six candidate decision families, a four-class constraint taxonomy, and the simulator–optimizer ownership boundary are fixed; the actual family, objective, and cadence wait for the Factory Tour (2026-09-11). All factory-specific quantities are labeled synthetic or unknown, and no claim is made about DENSO’s actual operations. A C++20 core at `cpp/` implements the feasibility semantics as the approved executable specification of this report.

Contents

1	Introduction	3
2	Scope, Assumptions, and Provenance	3
3	Problem Definition	3
3.1	States, Actions, and the Catalog	3
3.2	Candidate Decision Families	4
3.3	Simulator Interface	5
4	Mathematical Model	5
4.1	Feasibility Semantics	6
4.2	Constraint Classes	7
4.3	Objective Structures	7
4.4	Uncertainty and Scenario Evaluation	8
5	Solution and Evaluation Strategy	9
5.1	Baselines	9
5.2	Exact Small-Instance Oracle	10
5.3	Simulation Evaluation Budget	10
5.4	Algorithm Selection and Implementation Status	10

6	Data and Validation Requirements	11
7	Factory-Tour Decision Gates	12
8	Limitations and Next Steps	12
A	Constraint Registry	13

1 Introduction

The D2 competition task requires a system that predicts future logistics state, detects bottlenecks, simulates what-if scenarios, and recommends feasible, useful actions. The conceptual pipeline is

factory state / forecast $\rightarrow$ **forecast** $\rightarrow$ **simulation** $+$ **bottleneck detection** $\rightarrow$
candidate actions $\rightarrow$ **feasibility** $\rightarrow$ **simulated KPI outcomes** $\rightarrow$ **ranking** $\rightarrow$
recommendation

This report specifies the decision layer only: given the current or forecast state and simulator outcomes, *which feasible action should be recommended, and on what evidence*. Two roles are separated sharply. The digital-twin/simulation layer (owned by Đào Văn Đức) models physical evolution, event dynamics, KPI generation, and bottleneck behavior. The decision layer (this report) owns candidate generation, feasibility modeling, objective interpretation, and ranking. The decision layer never re-creates simulator semantics, and the simulator never selects the recommended action ([Section 3.3](#)).

The report is deliberately written *before* the Factory Tour (2026-09-11). Its purpose is to fix what can be modeled safely now — the feasibility semantics, the constraint taxonomy, the candidate decision families, the objective structures, and the evaluation methodology — and to make explicit which facts must come from the plant before a final formulation, objective, or algorithm can be chosen. Every factory-specific quantity in this report is synthetic (an invented development value) or unknown (missing information that blocks the claims depending on it); none is a claim about DENSO’s operations.

2 Scope, Assumptions, and Provenance

Two provenance vocabularies are used, and they are not interchangeable. The *data-parameter source types* are owned by the data layer and used verbatim here:

$$\mathcal{S}_{\text{src}} = \{\text{synthetic, observed, estimated, assumed, mentor_confirmed, system_export}\}. \quad (1)$$

Separately, this report attaches *claim statuses* to its own modeling statements ([Table 1](#)). Each parameter records a tuple $\langle \text{value, unit, source, confidence, validation status} \rangle$; a parameter whose value, unit, or source is not established is unknown and must not silently become a numerical assumption.

The central discipline of this report is the unknown-information rule: missing information about a required hard constraint yields status unknown, never implicit feasibility,

$$\text{data missing for constraint } i \implies \phi_i(a, s_t) = \text{unknown}, \quad a \notin \mathcal{A}_F(s_t), \quad (2)$$

i.e., the affected action is blocked and flagged for mentor validation rather than passed. This is the opposite of the common failure mode in which absent constraints silently make every action feasible.

3 Problem Definition

3.1 States, Actions, and the Catalog

Let s_t denote the decision-relevant state at epoch t , assembled from simulation outputs (queue lengths, WIP, utilization, bottleneck label) and the forecast trajectory. The decision layer proposes actions from a state-dependent set $\mathcal{A}(s_t)$, drawn from an *action catalog* whose entries

Table 1: Claim statuses and permitted use. Data-parameter provenance uses the data-layer enum (1), not this table.

Status	Meaning	Can support a DENSO claim?
GENERIC	Structural property valid across factories (capacities are finite)	No — structure only
SYNTHETIC	Invented value for development/testing (e.g., 2 transporters)	No — development only
CANDIDATE	Proposed structure awaiting Factory-Tour validation	No — design only
OBSERVED	Measured during the tour with provenance	Provisionally, with sample basis
MENTOR-CONFIRMED	Validated by a DENSO mentor	Yes
UNKNOWN	Required information not yet available	No — blocks dependent claims

carry provenance (source type, confidence, validation status, mentor flag) and are, before the tour, synthetic development artifacts. The catalog contains the explicit no-action alternative a_0 ; “no feasible improvement” is a valid, reportable outcome of every decision epoch, not a failure.

Six generic action families are CANDIDATE structures for the catalog — resource reallocation, dispatch-priority change, route change, buffer/capacity adjustment, replenishment-interval change, and temporary capacity addition — alongside the explicit no-op (a_0 itself). These *action families* (the typed payload shapes realized in the C++ core) must not be confused with the *decision families* of Section 3.2, which group whole decision problems. Concrete action semantics (which resource, which route, which quantity) are unknown until the tour; the C++ core fixes only the typed payload shapes, not any DENSO instances.

3.2 Candidate Decision Families

The actual DENSO decision family is unknown before the tour. Six families are formulated as candidates (Table 2); structural taxonomies of automotive intralogistics support the family menu [5, 13, 8], but none of them is evidence about DENSO’s plant.

Resource reallocation and dispatching carry a modest catalog-informed prior: one of the two synthetic development actions is a resource-count change, and dispatch-rule selection is the other lever the current pipeline can express. For reallocation, the minimal structure is an assignment problem: with $x_{rj} = 1$ iff resource r serves job j , resource exclusivity and job coverage read

$$\sum_{j \in J} x_{rj} \leq 1 \quad \forall r \in R, \quad \sum_{r \in R} x_{rj} = 1 \quad \forall j \in J, \quad (3)$$

which becomes a generalized assignment problem once capacity or travel-time costs enter. For dispatching, the rule π is chosen from a small catalog Π (FIFO, earliest-deadline-first, bottleneck-first); at single-digit catalog sizes, exhaustive evaluation of all rules is both optimal within the catalog and fully explainable, so sophisticated heuristics add nothing.

The routing family deserves one caution: the AGV/AMR literature [13, 8] is an analogue, not proof that D2 is an AGV routing problem — if the tour confirms fixed routes and tugger-train operations, the family collapses into replenishment. The integrated production–transport family is conditional on tour question S7; published approaches at scale are reported to use matheuristics or decomposition, but those pointers are unverified in our survey and no claim

Table 2: Candidate decision families. All remain CANDIDATE until the Factory Tour; the priors on the first two rows are inferences from our own synthetic 2-action development catalog, not evidence about DENSO.

Family	Decision ables	vari-	Data needed	First method	Tour trigger
Resource reallo- cation	$x_{r,j} \in \{0,1\}$		counts, ceilings compatibility	$\bar{c}_r$, assignment / MILP	resources re- assignable at operational ca- dence
Dispatching	$\pi \in \Pi$		current rule, allowed variants	catalog enumera- tion	dispatch rules ad- justable
Routing	$y_{r,e,t} \in \{0,1\}$		route graph, travel times	exact + local search	routing freedom confirmed
Replenishment	ρ_j, β_j		policy bounds, de- mand link	policy enumeration / MILP	line feeding con- firmed
Buffer / capacity	$\Delta C_n, u_r$		limits, authorization, cost	small integer set	buffer adjust- ment executable
Integrated prod.– transport	coupled schedul- ing variables		schedule interface, coupling	exact small / matheuristic	strong coupling (S7)

here is source-supported. It is substantially harder, because production and transport variables must be coupled over time, and should be selected only if the coupling is real.

3.3 Simulator Interface

The simulator is modeled as a map

$$Y : (s_t, a, \xi_s, \omega) \mapsto \mathbf{v} \in \mathbb{R}^K, \quad (4)$$

returning the KPI vector realized under action a , scenario ξ_s , and replication seed ω . The decision layer consumes `SimulationResult` objects only through this contract — with per-run metadata (scenario ID, seed, KPI units, action ID) — and all physical semantics stay behind it.

Decision layer owns	Simulation layer owns
Candidate generation ($\mathcal{A}(s_t)$)	Physical evolution of the system
Feasibility filtering (ϕ_i , verdicts)	Event dynamics, queues, capacities
Objective interpretation (F)	KPI value generation (definitions per Red deci- sion)
Ranking and recommendation	Bottleneck detection behavior
Provenance labels on recommendations	Seeds, streams, replications

The decision layer must not re-create simulator semantics, and the simulation layer must not silently select the final recommended action. This split mirrors the constraint-split architecture verified in recent simulation-optimization practice [9]: the optimizer enforces deterministic feasibility and proposes partial decisions; the simulator completes them and evaluates outcomes. Action masking under a formal system model is the same principle [11].

4 Mathematical Model

Table 3 fixes the notation. Sets R, N, E, J are generic structures; their instances (which resources exist, which routes are admissible) are unknown until the tour, and no set is instantiated before its family is selected.

Table 3: Notation. Only symbols used in this report are listed.

Symbol	Meaning
N, R, J, E	Nodes (stations, buffers, supermarkets); resources (transporters, AGVs, operators); jobs; admissible routes
$A, \mathcal{A}(s_t)$	Action catalog; actions applicable in state s_t
T^{ep}, t, Δ	Decision epochs, epoch index, epoch length; horizon $H = T^{\text{ep}} $
$\mathcal{S}, \xi_s$	Uncertainty scenarios; joint trajectory (forecast path + parameter set)
ω	Simulation replication seed
K	KPI dimensions
$\mathcal{H}, g_i, \phi_i$	Hard constraints; mathematical form $g_i \leq 0$; three-valued status function
s_t	System state at epoch t (queues, WIP, utilization, bottleneck label, forecast trajectory)
$c_r, \delta_r, \bar{c}_r$	Current count of resource r ; count change proposed by an action; pool ceiling
$C_n, q_n(t)$	Capacity of node n ; queue/WIP content at node n
τ_e, p_j	Travel time on route e ; processing time of job j
c_a, B	Implementation cost of action a ; per-epoch budget
$x_{rj}, \pi, y_{r,e,t}, \rho_j, \beta_j, u_r, \Delta C_n$	Family decision variables (Table 2)
$Y, \mathbf{y}(a), \Delta_s(a)$	Simulator map (4); KPI outcome vector; KPI delta vs. no-action under scenario s
F, a^*	Preference functional; recommended action

4.1 Feasibility Semantics

Each hard constraint $i \in \mathcal{H}$ carries a status function

$$\phi_i(a, s_t) \in \{\text{satisfied}, \text{violated}, \text{unknown}\}, \quad (5)$$

mapping the mathematical form $g_i(a, s_t) \leq 0$ to **satisfied** when g_i is verified evaluable with $g_i(a, s_t) \leq 0$, to **violated** when verified evaluable with $g_i(a, s_t) > 0$, and to **unknown** when the data required to evaluate g_i is missing. The action-level verdict composes these values with three-valued logic:

$$\text{verdict}(a) = \begin{cases} \text{INFEASIBLE} & \exists i \in \mathcal{H} : \phi_i(a, s_t) = \text{violated}, \\ \text{UNKNOWN} & \phi_i(a, s_t) \neq \text{violated} \forall i \in \mathcal{H} \text{ and } \exists i \in \mathcal{H} : \phi_i(a, s_t) = \text{unknown}, \\ \text{FEASIBLE} & \phi_i(a, s_t) = \text{satisfied} \forall i \in \mathcal{H}. \end{cases} \quad (6)$$

Violation dominates; unverifiable constraints surface as unknown rather than being silently passed; only fully verified satisfaction yields feasibility. The feasible set is therefore

$$\mathcal{A}_F(s_t) = \{a \in \mathcal{A}(s_t) : \phi_i(a, s_t) = \text{satisfied} \quad \forall i \in \mathcal{H}\}. \quad (7)$$

The no-action alternative a_0 is feasible *by construction* in the following evidence-aware sense. Change-dependent constraints take their $\delta = 0$ branch, so unknown data about proposed changes (pool ceilings, budgets, lead times) never blocks the status quo. State-dependent constraints, however, evaluate a_0 for real: if verified state evidence shows the status quo already violates such a constraint (an evidenced buffer overflow), the violation is reported for a_0 exactly as Equation (5) requires — satisfaction is never fabricated from missing change data, and known violations are never suppressed.

Soft constraints do not participate in the verdict of (6). A constraint may be declared soft by a named owner; its violation then appears as an explicit flag in the recommendation reason, never as a silent score shift, and hard-constraint violations are never expressed as objective penalties. One metamorphic property is used throughout the pre-tour test suite: adding hard constraints (in particular, tightening one) cannot enlarge the feasible set,

$$\mathcal{H}' \supseteq \mathcal{H} \implies \mathcal{A}_F(s_t; \mathcal{H}') \subseteq \mathcal{A}_F(s_t; \mathcal{H}). \quad (8)$$

With feasibility fixed, the recommendation problem is

$$a^* \in \arg \min_{a \in \mathcal{A}_F(s_t)} F(\{\Delta_s(a)\}_{s \in \mathcal{S}}), \quad \Delta_s(a) := Y(s_t, a, \xi_s, \omega) - Y(s_t, a_0, \xi_s, \omega), \quad (9)$$

where F is a preference functional over scenario-indexed KPI deltas (Section 4.3). Deliberately, (9) does not commit to a scalar objective, and selecting F (weights, orders, thresholds) is a human decision, not an agent inference.

4.2 Constraint Classes

Constraints fall into four classes: *physical* (the modeled system itself; semantics owned by the simulation layer), *operational* (how the plant is run), *business* (budget, labor, service commitments), and *decision-policy* (self-governance rules of the decision layer itself, e.g. “no action is labeled feasible with unvalidated provenance”). The full pre-tour registry with identifiers, forms, and data needs is in Appendix A; two structural aspects belong here.

First, every constraint carries a *check layer*: *static* (evaluable from configuration before simulation), *simulator-enforced* (an invariant the simulator must maintain — resource exclusivity, buffer bounds, flow conservation, availability windows — where the decision layer can only report violations observed in simulation results), or *dynamic* (checked on simulation outputs). For a simulator-enforced constraint, the decision layer does not evaluate ϕ_i from configuration; it remains **unknown** unless simulation evidence is supplied.

Second, static pre-checks are admissible in one direction only. When the *proposed change* alone makes satisfaction impossible — an `ADJUST_BUFFER` action requesting a negative target level, or a proposed resource count exceeding a known pool ceiling, $0 \leq c_r + \delta_r \leq \bar{c}_r$ — the decision layer may statically detect the violation at proposal time. Such checks turn **unknown** into **violated**, never into **satisfied**: verifying that a simulator-enforced constraint *held* requires simulation evidence. C-OPS-007 (action lead time) similarly admits a static deadline check whenever the epoch carries a known response deadline.

4.3 Objective Structures

The KPI set is provisional; each KPI must eventually carry definition, unit, aggregation window, improvement direction, and DENSO priority, none of which is confirmed. The candidate KPI vector (deltas relative to no-action) is

$$\mathbf{y}(a) = [\Delta \text{ throughput}, \Delta \text{ lead time}, \Delta \text{ lateness}, \Delta \text{ WIP}, \Delta \text{ cost}, \text{risk}], \quad (10)$$

where risk is a placeholder for scenario-spread summaries. Queue length, service level, and stability may join the vector after tour confirmation; utilization is deliberately excluded, because high utilization at a constraint is efficient while at non-constraints it signals fragility — its direction is context-dependent and its priority is a DENSO decision.

Two preference structures are the primary candidates. The constrained single objective,

$$\min_{a \in \mathcal{A}_F(s_t)} c_a \quad \text{s.t.} \quad L(a) \leq L_{\max}, \quad (11)$$

is appropriate when one operational limit dominates and the threshold $L_{\max}$ is defensible; the threshold itself is a human decision. The lexicographic objective orders KPIs by strict priority $k_1 \succ k_2 \succ \dots$: action a dominates b if

$$f_{k_1}(a) < f_{k_1}(b) \quad \text{or} \quad (f_{k_1}(a) = f_{k_1}(b) \text{ within tolerance } \varepsilon_1 \text{ and } (f_{k_2}(a), \dots) \succ_{\text{lex}} (f_{k_2}(b), \dots)). \quad (12)$$

Tolerances ε_k must be explicit; without them, negligible primary gains can justify large secondary losses. Lexicographic methods are standard preference-modeling tools [15]; whether DENSO priorities are genuinely strict is a tour question.

Table 4: Alternative preference structures, kept as candidates. None is adopted before the tour.

Structure	Form	Appropriate when	Caution
Epsilon-constraint	$\min f_1 \text{ s.t. } f_k \leq \epsilon_k$	thresholds are operationally interpretable	thresholds are policy decisions; infeasible combinations must be reported [15]
Weighted normalized	$F = \sum_k w_k \hat{f}_k$	preferences are genuinely compensatory and weights are elicited	incompatible units; scale sensitivity; unstable under $\pm 20\%$ weight perturbation
Pareto screening	dominance over $\mathbf{y}(a)$	first screen before weights exist	a frontier is a shortlist, not a recommendation; noise corrupts dominance at small sample sizes

Evolutionary frontier search such as NSGA-II [7] is justified only when enumeration is impractical and noise is controlled; with tens of candidates and noisy simulation outputs, enumeration with replication discipline is usually superior. Risk-aware aggregation is presented but deferred: given scenario losses $C_s(a)$, the candidates are the expectation $\mathbb{E}[C(a)] = \sum_s p_s C_s(a)$, the worst case $\max_s C_s(a)$, and the CVaR representation [16]

$$\text{CVaR}_\alpha(C(a)) = \min_t \left[t + \frac{1}{1-\alpha} \mathbb{E}[(C(a) - t)_+] \right]. \quad (13)$$

Escalating a risk measure from a reported column to a decisive rule requires (i) asymmetric downside consequences, (ii) service-level violations that matter, and (iii) sufficient tail data — empirical CVaR at level α from N samples is driven by only $\approx N(1 - \alpha)$ tail observations. None of the three holds today: the pipeline is deterministic. The pre-tour policy is report-first — rank on expected KPI, report observed worst case and (when tail support suffices) CVaR as secondary columns.

Cost semantics deserve one explicit rule: *known-zero cost and unknown cost are distinct states*. An action whose cost has not been recorded is not a free action; ranking rules may consume cost only when its value and unit are known (mentor-confirmed or synthetic by design). If absolute costs stay undisclosed, an explicitly labeled ordinal ranking (cheap/medium/expensive) is acceptable, and the objective must then use forms that do not need a common unit — lexicographic or epsilon-constraint, never a fabricated currency sum.

4.4 Uncertainty and Scenario Evaluation

Three uncertainty sources are kept distinct: (i) *forecast uncertainty* — the forecast contract provides quantile paths (p10/p50/p90; current code consumes only p50), and quantiles are not scenarios with probabilities; (ii) *process randomness* — service, travel, and breakdown variability inside the simulator, which exists only once the DES is stochastic; and (iii) *parameter uncertainty* — tour-measured values may be wrong or drift, handled by labeled ranges and sensitivity analysis. Forecast residual uncertainty and simulator variability can model the same mechanism twice; scenario construction must state which variability lives where. Constructing valid joint scenario trajectories is a forecast-layer task: trajectories must preserve serial correlation across the horizon and cross-variable dependence (independent per-period sampling from marginals creates impossible trajectories), and any scenario weights require a sourced justification.

Actions are compared on a *shared scenario–seed bank*: the identical scenario set and seed set across actions wherever possible. The same integer seed does not by itself guarantee valid pairing once actions alter event paths, so pairing requires either a common-random-numbers mechanism (a simulator-design question owned by the simulation layer) or adequate independent replications with explicit unpaired analysis. Reporting uses mean, variance, quantiles, and confidence intervals [2]. Quantile paths are used as labeled stress tests — a candidate is evaluated under each path and rank stability across paths is reported — never averaged into a single number, which would manufacture a probability interpretation the forecast never provided. Sample-average approximation formalizes Monte Carlo evaluation of expectations when scenario sets are sampled [3, 10]; robust-optimization and scenario-approach guarantees are explicitly not transferred to our black-box DES setting without their own assumptions [4, 6].

5 Solution and Evaluation Strategy

5.1 Baselines

Table 5: Decision-method ladder. Every stronger method must beat the mandatory baselines under the same scenario–seed bank before it is considered.

Method	Role	Placement rationale
Baseline 0: no action	Null recommendation; every claim is a delta vs. this	Mandatory reference
Baseline 1: rule-based heuristic	“if bottleneck = transport, add transport capacity”-style rules	Mandatory; simple, explainable
Baseline 2: greedy enumeration	Evaluate all feasible single actions; rank by approved rule	Likely first real optimizer; optimal within the catalog
Exact reference	Exhaustive enumeration on tiny instances	Correctness oracle (golden cases)
Structured exact	MILP / CP-SAT / min-cost flow / assignment	Only if the family has algebraic structure
Conditional larger search	Local search, tabu, ILS, ALNS	Only when $ \mathcal{A} \gg \text{budget}$ and the family survives selection
Deferred	NSGA-II, RL, matheuristics, DRO	Not justified pre-tour (catalog small; assumptions unverified)

Selection rule (pre-tour default). The Baseline 2 pipeline recommends a candidate only when it is *strictly* preferred over the no-action alternative a_0 under the declared policy: a tie with a_0 keeps the status quo, because a zero-improvement change with unknown cost and lead time is not a recommendation. a_0 itself is evaluated through the same gate; when evidence shows the status quo violates a hard constraint, a_0 is infeasible — it is then neither simulated nor compared against, and the best feasible candidate is recommended while the a_0 entry exposes the violation. When nothing is feasible, the outcome is an explicit no-feasible-option report, never a fabricated recommendation. This rule is a conservative pre-tour default and is a Factory-Tour confirmation item, not a settled preference.

For small discrete candidate sets, enumeration with statistically careful comparison is the methodologically indicated choice [3, 10]; recent results show that for small recurring re-decisions even exact solutions can be milliseconds away [1], so search sophistication is not

the bottleneck at our scale. Ranking-and-selection machinery for many noisy alternatives is deferred until needed [17].

One capability gap is acknowledged plainly: the current repository simulator accepts only a configuration and a forecast — it cannot accept an action — so simulation-based ranking is blocked until the simulation owner delivers an action-capable simulator. Until then, the implementable decision-layer work is the feasibility gate, the provenance-labeled action catalog, and ranking over synthetic analytic outcomes, never fabricated simulation results.

5.2 Exact Small-Instance Oracle

For small synthetic instances (2–3 resources, 3–5 actions), all feasible single actions are enumerated and the exact optimum f^* of the model under evaluation is computed. A ranking method produces f_h ; the optimality gap is

$$\text{gap}(f_h, f^*) = \begin{cases} \frac{f_h - f^*}{|f^*|}, & f^* \neq 0, \\ f_h - f^*, & f^* = 0 \quad (\text{zero-denominator guard}). \end{cases} \quad (14)$$

This is a correctness benchmark on synthetic instances only — never a DENSIO performance result. Its purposes are to verify that feasibility, ranking, and preference-rule modules reproduce hand-computable optima, to characterize runtime scaling honestly, and to serve as a differential-testing oracle for any future optimized search kernel.

5.3 Simulation Evaluation Budget

The evaluation cost per decision epoch is approximately

$$\text{Cost} \propto |\mathcal{A}| \times |\mathcal{S}| \times R \times t_{\text{sim}}, \quad (15)$$

with R replications per (action, scenario) and t_{sim} runtime per run. With a toy simulator and a small catalog the cost is negligible; it becomes binding once the product reaches the order of hundreds to thousands of full runs per epoch (e.g., $20 \times 10 \times 5 = 1000$), or when t_{sim} grows with a real DES. The first response is not optimization but the two-tier budget pattern verified in recent simulation optimization [9]: a small replication budget per candidate during ranking ($R = 5\text{--}10$; design proposal, not a universal constant), followed by a larger *fresh-sample* validation of the selected finalist ($\geq 10R$) that is never tuned on and used only for confirmation. Sequential elimination [17], budget allocation inside metaheuristics, common-random-numbers variance reduction, memoization, and surrogates are deferred until a seeded DES and a measured need exist.

5.4 Algorithm Selection and Implementation Status

Algorithm selection follows problem identification, never precedes it. Confirmed problem shape maps to a first method family: a small action catalog maps to enumeration/greedy with paired evaluation; structured assignment to MILP/CP-SAT/min-cost flow; routing and scheduling to an exact baseline plus neighborhood search; a large black-box DES problem to simulation optimization with a fixed budget and final validation; a dynamic decision cadence to rolling-horizon or event-triggered re-solving. MILP, CP-SAT, tabu, ILS, ALNS, genetic algorithms, and reinforcement learning are not selected prematurely, and nothing native is justified at current problem sizes.

Python remains the reference language for the pipeline (model, baselines, orchestration, small exact instances); native kernels are a post-profiling decision — C++ when the solver ecosystem, combinatorial structure, or team familiarity dominates, Rust when a larger custom

parallel search engine with concurrency-safety requirements is warranted. No native *search kernel* is planned in the competition window. A C++20 core at `cpp/` implements the feasibility semantics of [Section 4.1](#) and the catalog as the approved executable specification of this report’s pre-tour semantics; that approval covers the executable-specification role, not a choice of C++ for future search kernels. Its action-cost handling deliberately differs from the Python shared contract — the C++ core omits cost entirely because its unit is unknown pre-tour, whereas the Python dataclass defaults `estimated_cost` to 0.0 — and reconciling the two is a shared-contract decision that must not be made silently by either layer.

6 Data and Validation Requirements

Table 6: Compact data-requirement map.

Field	Used by	Required	Unit	Status / tour need
Resource IDs	assignment family	yes	—	UNKNOWN (structure only)
Route topology	routing, connectivity	yes	graph	UNKNOWN
Travel time τ_e	simulation, actions	yes	min	SYNTHETIC
Loading/unloading time	simulation	yes	min	UNKNOWN
Resource count c_r	feasibility C-PHY-002	yes	count	SYNTHETIC (2)
Pool ceiling $\bar{c}_r$	ADD_RESOURCE bound	yes	count	UNKNOWN
Buffer capacity C_n	C-PHY-003	yes	units	SYNTHETIC (20)
Queue/WIP q_n	state s_t	runtime	jobs	SYNTHETIC (toy simulator)
Demand forecast	scenario construction	yes	jobs/h	SYNTHETIC (quantiles)
Replenishment policy	replenishment family	family	min/units	UNKNOWN
Deadline / service target	lateness KPI	objective	time	UNKNOWN
Shift schedule	C-OPS-002, epochs	yes	schedule	UNKNOWN
Action catalog	decision layer	yes	—	SYNTHETIC (2 actions)
Action cost c_a	C-BUS-001/003	ranking	currency/ordinal	UNKNOWN
Action lead time	C-OPS-007	honest ranking	min	UNKNOWN
Decision cadence	epoch design T^{ep}	yes	per shift/day	UNKNOWN
KPI priorities	objective F	yes	—	UNKNOWN (Red decision)

Validation proceeds in three layers before any optimization run: (A) schema validation (fields, types, units, referential integrity); (B) domain/physical checks — non-negativity, capacity bounds, utilization in $[0, 1]$, interval ordering — which are machine-checkable now; and (C) statistical/temporal checks (distributions, gaps, drift, leakage) once enough data exists. Impossible records are flagged, never silently repaired. One reporting rule accompanies the KPI units once confirmed: a delta smaller than the evaluation noise floor is not an improvement — before an action is called better than no-action, its paired delta must exceed the dispersion of the paired differences (e.g., the confidence interval must exclude zero) and matter operationally.

Synthetic development scenarios prove *algorithmic* properties — correctness of feasibility logic, contract shapes, invariants, deterministic behavior (same config + seed $\Rightarrow$ same result),

and expected directional responses — and cannot prove real DENSO accuracy, ROI, bottleneck identity, or optimality of any action. The six named scenarios are **balanced_flow** (no bottleneck; no action claims large improvement), **known_transport_bottleneck** (transporter-constrained state; a feasible **ADD_RESOURCE** exists), **buffer_overflow** (queue at/over capacity; buffer-family and hard-constraint tests), **demand_spike** (p90 trajectory stress; capacity interaction), **resource_shortage** (pool ceiling makes **ADD_RESOURCE** infeasible via C-PHY-002), and **all_actions_infeasible** (the valid output is an empty list with named violations, not a fake recommendation). Metamorphic properties tested on them include Equation (8); increasing bottleneck capacity must not decrease throughput in a deterministic scenario; adding a dominated action must not change the winner; and missing constraint data must block, not pass, the affected action.

7 Factory-Tour Decision Gates

The Factory Tour (2026-09-11) resolves the core unknowns. Each question maps to a specific part of the model:

Question	Affects
M1. Actual logistics pain point?	Family selection (Section 3.2); everything downstream
M2. Which 2–3 actions are executable?	Action space $\mathcal{A}(s_t)$, action catalog; feasibility data
M3. Which KPIs matter; current values?	Objective F (Section 4.3); improvement sizing
M4. What is forbidden?	Hard-constraint set $\mathcal{H}$ (Section 4.1, Appendix A)
M5. Decision cadence / response deadline?	Static vs. rolling-horizon formulation; epoch design T^{ep} ; runtime budget
M6. One historical incident walkthrough	Ground-truth anchor for action effects; validation
S1. Current manual decision rule?	Baseline 0/1 definition (Section 5.1)
S2. Cost structure (even ordinal)?	C-BUS constraints; objective cost term (Section 4.3)
S3. Resource flexibility / pool ceiling $\bar{c}_r$?	C-PHY-002 bound; resource-family viability
S5. Action lead times / stability?	C-OPS-007/008 data; ranking honesty
S7. Production–transport coupling?	Whether the integrated family (Section 3.2) is live; simulator interface granularity
S8. Re-decision trigger in practice?	Rolling-horizon vs. event-triggered vs. one-shot; epoch design (refines M5)
S4 (data availability/latency) and S6 (demand pattern/forecast practice) feed the data and forecast layers, not the decision model directly; they are captured in the tour checklist but omitted here.	

After the tour, each family is scored on relevance, data availability, actionability, optimization structure, and simulator compatibility, with a Go/No-Go gate per family. Bottleneck detection research [14, 12] motivates the cadence questions: bottlenecks shift over time, so a one-shot formulation is unlikely to survive contact with operations. All post-tour selection statuses are provisional by construction until the tour answers arrive.

8 Limitations and Next Steps

What this report fixes: the decision-layer architecture (generate $\rightarrow$ gate $\rightarrow$ simulate $\rightarrow$ rank $\rightarrow$ recommend), the three-valued feasibility semantics (6), the four-class constraint taxonomy with check layers, the candidate objective structures with the preference rule left as a human decision, the simulator–optimizer ownership boundary, and the evaluation methodology (shared scenario–seed banks, paired deltas, two-tier replication budget, fresh-sample confirmation).

What remains unknown: the actual decision family, the executable action space, factory-specific hard constraints, the KPI hierarchy, decision cadence, and real cost parameters — all gated on the Factory Tour. The report’s honest limit is that it is a pre-tour artifact: it defines how a recommendation must be produced and evaluated, not what DENSO’s plant will actually require.

The Baseline 0–2 pipeline is implemented and tested in the C++ core (`‘cpp/’`): an explicit no-action baseline evaluated through the same simulation boundary as every candidate, a generic rule-based candidate generator, and greedy enumeration under an explicitly synthetic test preference policy — differential-tested against a tiny exhaustive oracle on the named synthetic scenarios. No final optimizer, heuristic, or native search kernel is attempted before the tour; the next integration step is an action-capable simulator from the simulation owner, so that candidates can be evaluated against a real Digital-Twin boundary instead of the current test double.

A Constraint Registry

This appendix lists the pre-tour constraint registry. Identifiers are stable and used by the C++ core; forms are generic. Statuses are claim statuses ([Table 1](#)).

Physical constraints

Table 7: Physical constraints. Simulation semantics owned by the simulation layer.

ID	Constraint	Mathematical form	Layer	Status
C-PHY-001	Resource exclusivity	$\sum_j x_{rj} \leq 1 \ \forall r$ (3)	sim-enf.	GENERIC
C-PHY-002	Resource-count bounds	$0 \leq c_r + \delta_r \leq \bar{c}_r$, pool ceiling UNKNOWN	static	GENERIC / HARD
C-PHY-003	Buffer bounds	$0 \leq q_n(t) \leq C_n \ \forall n, t$	sim-enf.	GENERIC / HARD
C-PHY-004	Flow conservation	inflow = outflow + Δ storage on every node	sim-enf.	GENERIC / HARD
C-PHY-005	Route connectivity	transport only along $e \in E$	static	GENERIC
C-PHY-006	Non-negativity	$\tau_e \geq 0, p_j \geq 0, q_n \geq 0$, WIP ≥ 0	static	UNIVERSAL
C-PHY-007	Availability windows	resource busy only within availability calendar	sim-enf.	UNKNOWN

Operational constraints

Table 8: Operational constraints (pre-tour candidates; most UNKNOWN).

ID	Constraint	Data needed	Status
C-OPS-001	Only catalog actions recommendable	action catalog w/ prove-nance	enforced now

ID	Constraint	Data needed	Status
C-OPS-002	Shift windows (changes at boundaries only)	shift schedule	UNKNOWN
C-OPS-003	Dispatch restrictions (fixed priorities)	dispatch rules	UNKNOWN
C-OPS-004	Replenishment policy bounds	policy parameters	UNKNOWN
C-OPS-005	Route/material restrictions	material-route compatibility	UNKNOWN
C-OPS-006	Safety: no unsafe congestion/interaction	layout, traffic rules	UNKNOWN
C-OPS-007	Action lead time (takes effect after implementation delay)	per-action lead time	UNKNOWN
C-OPS-008	Action cooldown / minimum stability period	cooldown policy	UNKNOWN

C-OPS-006 is hard by default: no action may create unsafe congestion (e.g., AMR–pedestrian interaction — synthetic illustration); its data is unknown.

Business constraints

ID	Constraint	Hard/soft	Status
C-BUS-001	Budget: $\sum_{a \in \text{selected}} c_a \leq B$	hard if budget real; else excluded	UNKNOWN
C-BUS-002	Labor limits (overtime, headcount)	hard	UNKNOWN
C-BUS-003	Implementation/switching cost counts	soft (flag) or objective term	design proposal
C-BUS-004	Maximum acceptable lateness/service level	hard if contractual	UNKNOWN

Decision-policy constraints (self-governance)

ID	Constraint	Status
C-POL-001	At most N_{chg} actions change per epoch ($\sum_a z_a \leq N_{\text{chg}}$)	N_{chg} UNKNOWN
C-POL-002	Recommendation must include no-action comparison	enforced (hard policy)
C-POL-003	No action labeled feasible with unknown/unvalidated provenance	enforced (hard policy)
C-POL-004	Combined/bundled actions disabled until single-action effects measured	enforced (hard policy, current)
C-POL-005	Preference rule fixed only by named human decision	enforced (hard policy)
C-POL-006	No DENSO-confidential data in Git/outputs/demos	enforced (hard policy)

References

- [1] Tommaso Adamo, Gilberto Ghiani, and Emanuela Guerriero. Real-time schedule adjustments for conflict-free vehicle routing. *Computers & Operations Research*, 166:106625, 2024.

- [2] Christos Alexopoulos. A comprehensive review of methods for simulation output analysis. In *Proceedings of the 2006 Winter Simulation Conference*, pages 168–178, 2006.
- [3] Satyajith Amaran, Nikolaos V. Sahinidis, Bikram Sharda, and Scott J. Bury. Simulation optimization: a review of algorithms and applications. *Annals of Operations Research*, 240(1):351–380, 2016.
- [4] Dimitris Bertsimas and Melvyn Sim. The price of robustness. *Operations Research*, 52(1):35–53, 2004.
- [5] Nils Boysen, Stefan Emde, Malte Hoeck, and Michel Kauderer. Part logistics in the automotive industry: Decision problems, literature review and research agenda. *European Journal of Operational Research*, 242(1):107–120, 2015.
- [6] Marco C. Campi and Simone Garatti. The exact feasibility of randomized solutions of uncertain convex programs. *SIAM Journal on Optimization*, 19(3):1211–1230, 2008.
- [7] Kalyanmoy Deb, Amrit Pratap, Sakshi Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.
- [8] Giada Fragapane, René de Koster, Fabio Sgarbossa, and Jan Ola Strandhagen. Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda. *European Journal of Operational Research*, 294(2):405–426, 2021.
- [9] Ehsan Ghaedy-Heidary, Ehsan Nejati, Ali Ghasemi, and S. Ali Torabi. A simulation optimization framework to solve Stochastic Flexible Job-Shop Scheduling Problems—Case: Semiconductor manufacturing. *Computers & Operations Research*, 163:106508, 2024.
- [10] Anton J. Kleywegt, Alexander Shapiro, and Tito Homem-de Mello. The sample average approximation method for stochastic discrete optimization. *SIAM Journal on Optimization*, 12(2):479–502, 2002.
- [11] Zeineb Lassoued, Laxmikant Shrikant Baheti, Lucia Weiß-Borkowski, Alexander Lier, and Andreas Schwung. Scheduling of flexible job shops with AGV transport and tool sharing using deep reinforcement learning. *Journal of Manufacturing Systems*, 82:405–419, 2025.
- [12] Sidney R. Lawrence and Arthur H. Buss. Shifting production bottlenecks: causes, cures, and conundrums. *Production and Operations Management*, 3(1):21–37, 1994.
- [13] Tuong Le-Anh and Marieke B. M. de Koster. A review of design and control of automated guided vehicle systems. *European Journal of Operational Research*, 171(1):1–23, 2006.
- [14] Li Li, Qing Chang, and Jun Ni. Data driven bottleneck detection of manufacturing systems. *International Journal of Production Research*, 47(18):5019–5036, 2009.
- [15] Kaisa Miettinen. *Nonlinear Multiobjective Optimization*. Springer, 1998.
- [16] R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *The Journal of Risk*, 2(3):21–41, 2000.
- [17] Yining Wang, Guangxin Wan, and Wei Chen. Bonferroni-free and indifference-zone-flexible sequential elimination for ranking and selection. *Operations Research*, 72(4):2119–2134, 2024.